Practica Final Integradora ADA

Universidad EAFIT

Analisis y Diseno de Algoritmos

David Guerra, Samuel Ramirez, Isabela Ruiz

Medellin, 2026

Introduccion

En esta practica final integradora se desarrollo un programa en C++ para aplicar diferentes tecnicas de diseno y analisis de algoritmos sobre el dataset `WA_Fn-UseC_-Telco-Customer-Churn.csv`. El trabajo se organizo en tres modulos principales: lectura y procesamiento inicial de datos, construccion de un grafo de infraestructura con arbol de expansion minima, y solucion de un problema de asignacion de ancho de banda mediante Mochila 0-1.

El objetivo general fue tomar los registros del dataset, almacenarlos en estructuras propias del programa y usarlos como base para resolver problemas distintos con enfoques algoritmicos diferentes. Para esto se implementaron un parser de CSV, un ordenamiento MergeSort, una busqueda binaria recursiva, un algoritmo de Kruskal con Union-Find y una solucion de programacion dinamica para Mochila 0-1.

Este informe presenta la documentacion final del proyecto, incluyendo la explicacion de los modulos implementados, las decisiones tecnicas principales, el analisis de complejidad y los resultados obtenidos en la ejecucion del programa.

Modulo A: Parser, MergeSort y Binary Search

Descripcion

El primer modulo se encarga de leer el dataset de clientes, extraer las columnas necesarias y preparar los datos para los algoritmos posteriores. El archivo usado como entrada es:

```
data/WA_Fn-UseC_-Telco-Customer-Churn.csv
```

Cada fila del CSV se representa mediante la estructura `Solicitud`, que almacena el identificador del cliente, su antiguedad en meses, el cobro mensual, el cobro total y el estado de churn. A partir de estas solicitudes se realiza un ordenamiento por `tenure` descendente y luego se ejecutan busquedas exactas sobre ese mismo campo.

Las columnas utilizadas del dataset son `customerID`, `tenure`, `MonthlyCharges`, `TotalCharges` y `Churn`. Las demas columnas no se usan en esta implementacion, porque los modulos del trabajo solo requieren antiguedad, cargos e informacion de permanencia del cliente.

Aspectos importantes del programa

La funcion principal de lectura es `parseSolicitudes`. Esta funcion recibe la ruta del archivo CSV y dos variables por referencia para guardar el total de registros leidos y la cantidad de valores vacios en `TotalCharges`. El parser no depende de posiciones fijas de columnas, sino que primero lee el encabezado del archivo y ubica cada columna por nombre. Esto permite que el programa sea menos fragil ante cambios en el orden de las columnas.

Para separar las lineas del CSV se implemento una funcion auxiliar que maneja comas, espacios al inicio y al final, y soporte basico para comillas dobles. Aunque el dataset no presenta casos complejos de texto con muchas comas internas, esta decision evita depender de un `split` demasiado simple.

Un caso especial se presenta con `TotalCharges`. En algunos registros este campo viene vacio o con espacios. Para evitar errores al convertirlo a numero, el programa guarda `0.0` y aumenta el contador `totalChargesNulos`. Con el dataset usado, se encontraron 7043 registros y 11 valores vacios en `TotalCharges`.

Despues de cargar los datos se implemento `mergeSortSolicitudes`, que ordena el vector de solicitudes por `tenure` en orden descendente. No se uso `std::sort` ni `std::stable_sort`, porque el objetivo era implementar el algoritmo. El MergeSort conserva estabilidad: si dos solicitudes tienen el mismo `tenure`, se mantiene primero la que venia antes en el vector original.

Finalmente, la busqueda se realiza con `findTenure`, una funcion que llama internamente a un Binary Search recursivo. Como el vector esta ordenado de forma descendente, la comparacion se ajusta a ese orden: si el valor de la mitad es mayor que el buscado, se continua por la derecha; si es menor, se continua por la izquierda. Si se encuentra una solicitud con el `tenure` solicitado, se retorna su indice; si no, se retorna `-1`.

El modulo genera dos archivos:

```
results/solicitudes_ordenadas.csv
results/busquedas_A.txt
```

El primero contiene las solicitudes ordenadas por `tenure` descendente y el segundo contiene los resultados de cinco consultas exactas por antiguedad.

Clasificacion del enfoque

Este modulo combina tres enfoques. El parser corresponde a un procesamiento secuencial de datos, porque recorre el archivo linea por linea. El ordenamiento corresponde a Divide and Conquer, ya que MergeSort divide el vector en mitades, ordena cada mitad recursivamente y luego mezcla los resultados. La busqueda binaria tambien usa una estrategia de division del espacio de busqueda, reduciendo el rango de candidatos en cada llamada recursiva.

Complejidad temporal y espacial

El parser recorre las filas del archivo una vez, por lo que su complejidad temporal es `O(n)`, donde `n` es el numero de registros. El espacio usado tambien es `O(n)`, porque se almacena un vector con todas las solicitudes.

MergeSort tiene complejidad temporal `O(n log n)`, debido a que divide el arreglo en niveles logaritmicos y en cada nivel mezcla todos los elementos. Su complejidad espacial es `O(n)`, porque utiliza un vector auxiliar para realizar las mezclas.

Binary Search tiene complejidad temporal `O(log n)`, porque en cada paso descarta aproximadamente la mitad del rango. Al estar implementado de forma recursiva, su espacio adicional es `O(log n)` por la pila de llamadas.

Componente	Tiempo	Espacio
Parser CSV	`O(n)`	`O(n)`
MergeSort	`O(n log n)`	`O(n)`
Binary Search recursivo	`O(log n)`	`O(log n)`
Escritura de CSV ordenado	`O(n)`	`O(1)` adicional

Pruebas y resultados

La ejecucion completa con el dataset incluido reporto los siguientes conteos:

Medida	Valor
Registros leidos	7043
`TotalCharges` vacios	11
Tamano del vector	7043
Lineas en `solicitudes_ordenadas.csv`	7044
`tenure` maximo despues de ordenar	72

Para revisar el comportamiento de MergeSort se midieron tres tamanos de entrada. Los tiempos dependen de la maquina y de la ejecucion, por lo que deben entenderse como una prueba experimental del ambiente usado:

Tamano `n`	Tiempo
7043	12.13 ms
3500	6.02 ms
1000	0.00 ms

El archivo de busquedas contiene las cinco consultas definidas para el modulo:

Consulta	Indice encontrado	`customerID`
`k = 72`	219	`6904-JLBGY`
`k = 60`	1429	`7426-WEIJX`
`k = 45`	2475	`7925-PNRGI`
`k = 30`	3410	`2684-EIWEO`
`k = 12`	4951	`7450-NWRTR`

Modulo B: Grafo de infraestructura y MST con Kruskal

Descripcion

El segundo modulo construye un grafo de infraestructura a partir de las solicitudes cargadas por el parser. La idea es agrupar los registros en 20 nodos de concentracion y calcular un costo de conexion entre cada par de nodos. Sobre ese grafo se aplica el algoritmo de Kruskal para obtener un arbol de expansion minima.

Los nodos se forman usando el indice de cada solicitud en el vector:

```
grupo = i % 20
```

Esto genera 20 grupos. Para cada grupo se calcula el promedio de `MonthlyCharges`, redondeado a dos decimales. Luego, para cada par de nodos `u` y `v`, se crea una arista cuyo peso es:

```
c(u, v) = floor(Mu + Mv)
```

Como se crea una arista entre cada par de nodos, el resultado es un grafo completo `K20`, con 20 nodos y 190 aristas.

Aspectos importantes del programa

La clase `Graph` contiene la informacion del grafo. Internamente almacena los promedios de cada grupo y el vector de aristas. La construccion se divide en dos pasos: primero se calculan los promedios `Mk` y despues se generan todas las aristas posibles entre pares de nodos.

La estructura `Edge` almacena los nodos `u`, `v` y el peso de la arista. Tambien define la comparacion por peso, necesaria para ordenar las aristas antes de ejecutar Kruskal.

Para evitar ciclos durante la construccion del MST se implemento la estructura `UnionFind`. Esta usa compresion de caminos en `find` y union por rango en `unite`. La compresion de caminos hace que los nodos visitados apunten directamente a

la raiz del conjunto, mientras que la union por rango evita que los arboles internos se vuelvan demasiado altos.

El algoritmo de Kruskal primero ordena las aristas por peso ascendente. Luego recorre ese orden y agrega una arista solo si conecta dos componentes distintas. Si la arista formaria un ciclo, se descarta. El proceso termina cuando se han seleccionado `n - 1` aristas.

El resultado se escribe en:

```
results/mst_red.txt
```

Clasificacion del enfoque

Este modulo usa un enfoque codicioso. Kruskal toma decisiones locales ordenando las aristas de menor a mayor peso y seleccionando aquellas que conectan componentes diferentes. La decision se justifica porque una arista de menor peso que cruza un corte puede pertenecer al MST, mientras que una arista que forma ciclo se descarta porque no ayuda a conectar nuevos componentes.

La estructura Union-Find complementa el enfoque codicioso, ya que permite consultar rapidamente si dos nodos pertenecen a la misma componente.

Complejidad temporal y espacial

La construccion del grafo requiere calcular promedios y generar todas las aristas. En esta instancia se tienen 20 nodos y 190 aristas. Para Kruskal, el costo dominante es ordenar las aristas, por lo que la complejidad temporal es `O(E log E)`. Las operaciones de Union-Find tienen costo amortizado casi constante, `O(alpha(V))`, donde `alpha` es la inversa de Ackermann.

Aspecto	Analisis	Resultado
Ordenamiento de aristas	`E` aristas con ordenamiento ascendente	`O(E log E)`
Union-Find	`find` y `unite` sobre las aristas	`O(E alpha(V))`
Tiempo total	Dominado por el ordenamiento	`O(E log E)`
Espacio	Aristas, promedios y Union-Find	`O(V + E)`

Pruebas y resultados

La salida generada reporta los siguientes datos del grafo:

Medida	Valor
Nodos	20
Aristas	190
Peso promedio de arista	129.04
Peso total del MST	2393.00
Aristas en el MST	19

Los promedios de `MonthlyCharges` por grupo fueron:

Grupo	Promedio USD	Grupo	Promedio USD
0	64.17	10	65.53
1	65.09	11	62.81
2	64.53	12	65.56
3	64.60	13	62.67
4	68.25	14	64.09
5	65.94	15	65.05
6	64.57	16	65.96
7	64.00	17	61.46
8	66.18	18	65.58
9	65.35	19	63.86

El nodo 17 tiene el menor promedio (`61.46 USD`), por lo que aparece como nodo central en el MST. Esto ocurre porque el peso de cada arista se calcula como la suma redondeada hacia abajo de los promedios, y conectar con el nodo de menor promedio tiende a producir aristas de menor costo.

Una parte de las aristas seleccionadas en el MST es:

#	Nodo u	Nodo v	Peso
1	11	17	124.00
2	13	17	124.00
3	2	17	125.00

#	Nodo u	Nodo v	Peso
4	17	19	125.00
5	7	17	125.00
6	14	17	125.00
7	0	17	125.00
19	4	17	129.00

Tambien se verifico manualmente el subgrafo inducido por los nodos `{0, 1, 2, 3, 4}`. Este subgrafo corresponde a `K5`, por lo que tiene 10 aristas. Kruskal selecciono 4 aristas, como corresponde a un MST de 5 nodos:

#	Nodo u	Nodo v	Peso
1	0	2	128.00
2	0	3	128.00
3	0	1	129.00
4	0	4	132.00

El peso total del MST del subgrafo fue `517.00`.

Propiedad de eleccion codiciosa

La correctitud de Kruskal se explica con la propiedad de corte y el lema del ciclo. Cuando una arista conecta dos componentes distintas, es candidata a ser incluida porque representa una conexion de bajo costo entre dos partes separadas del grafo. Cuando una arista conecta dos nodos que ya estan en la misma componente, incluirla formaria un ciclo, por lo que se descarta.

En el grafo generado, una arista concreta seleccionada fue `(11, 17)` con peso:

```
floor(62.81 + 61.46) = 124
```

Al procesarla, los nodos 11 y 17 estaban en componentes distintas. Por eso se incluyo en el MST. Reemplazar esa conexion por otra de mayor peso aumentaria el costo total, lo que contradice el objetivo de obtener un arbol de expansion minima.

Modulo C: Asignacion de ancho de banda con Mochila 0-1

Descripcion

El tercer modulo aplica el problema de Mochila 0-1 a una asignacion de ancho de banda. A partir del vector de solicitudes ya ordenado por `tenure` descendente, se seleccionan las primeras 50 solicitudes activas, es decir, aquellas con `Churn = No`. Cada solicitud se transforma en un item de mochila.

Para cada item se define:

```
w_i = round(TotalCharges)
v_i = round(MonthlyCharges * 10)
```

La capacidad usada es:

```
W = 500
```

El objetivo es maximizar el valor total sin exceder la capacidad. Como cada solicitud puede tomarse o no tomarse, y no se permiten fracciones, el problema corresponde a Mochila 0-1.

Aspectos importantes del programa

La funcion `construirItems` recorre las solicitudes ordenadas y toma las primeras 50 con `Churn = No`. Cada una se convierte en un `ItemMochila`, que contiene el `customerID`, el peso `w` y el valor `v`.

La funcion `knapsack01` construye una tabla `dp` de dimensiones `(n + 1) x (W + 1)`. Cada celda `dp[i][w]` representa el valor maximo que se puede obtener considerando los primeros `i` items y una capacidad `w`.

La recurrencia usada es la clasica de Mochila 0-1. Si el peso del item actual supera la capacidad disponible, no se puede incluir. Si cabe, se toma el maximo entre no incluirlo e incluirlo:

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w-w_i] + v_i)
```

Despues de construir la tabla, la funcion `backtrack` reconstruye la solucion desde `dp[n][W]`. Si `dp[i][cap]` es diferente de `dp[i-1][cap]`, significa que el item `i` fue incluido, y entonces se descuenta su peso de la capacidad restante.

Los resultados se escriben en:

```
results/asignacion_bw.txt
```

Clasificacion del enfoque

Este modulo usa programacion dinamica por tabulacion. La decision de usar DP se debe a que Mochila 0-1 no se resuelve de forma exacta con un criterio codicioso general. La tabla permite guardar soluciones optimas de subproblemas y reutilizarlas para construir la solucion final.

El algoritmo es pseudopolinomial. Aunque su tiempo es proporcional a `n * W`, el valor `W` no representa directamente el tamano binario de la entrada. Por eso la Mochila 0-1 sigue siendo un problema NP-dificil en general, aunque esta solucion es eficiente para la capacidad concreta usada.

Complejidad temporal y espacial

Con `n = 50` y `W = 500`, la tabla tiene 51 filas y 501 columnas. La construccion de la tabla calcula una celda en tiempo constante, por lo que el tiempo total es `Theta(n * W)`.

Funcion	Tiempo	Espacio	Observacion
`construirItems`	`O(m)`	`O(50)`	`m <= 7043` registros recorridos
`knapsack01`	`Theta(n * W)`	`Theta(n * W)`	Dominante del modulo
`backtrack`	`O(n)`	`O(n)`	Recupera indices seleccionados
Escritura de resultados	`O(n)`	`O(1)` adicional	Salida secuencial

Para esta instancia:

```
Tiempo: 50 x 501 = 25050 celdas
Espacio: 51 x 501 = 25551 enteros
```

Pruebas y resultados

La ejecucion genero la tabla de programacion dinamica, pero el resultado optimo fue vacio:

Medida	Valor
Items	50
Capacidad `W`	500
Dimensiones de la tabla	51 × 501
Peso minimo entre los 50 items	1363
Valor optimo `dp[50][500]`	0
Solicitudes seleccionadas	0

Este resultado no corresponde a un error de implementacion. La razon es que el menor peso entre las 50 solicitudes seleccionadas es 1363, y todos los pesos superan la capacidad 500. Por lo tanto, ningun item cabe en la mochila y el valor optimo correcto es 0.

Contraejemplo al enfoque codicioso

Para mostrar que un enfoque codicioso por ratio `v_i / w_i` no siempre produce la solucion optima en Mochila 0-1, se uso un subproblema con tres solicitudes. La capacidad del subproblema fue:

```
W' = 1363 + 1469 = 2832
```

Item	`customerID`	`w_i`	`v_i`	Ratio `v/w`
A	`4312-KFRXN`	1711	254	0.148451
B	`2848-YXSMW`	1363	194	0.142333
C	`7606-BPHHN`	1469	198	0.134786

El codicioso selecciona primero A porque tiene el mayor ratio. Despues de tomar A queda una capacidad de 1121, insuficiente para B o C. El valor total seria 254. En cambio, la programacion dinamica selecciona B y C, que juntos tienen peso 2832 y valor 392.

Enfoque	Solicitudes seleccionadas	Valor total	Optimo
Codicioso por ratio	A	254	No
Programacion dinamica	B + C	392	Si

Esto demuestra que el criterio local de mayor ratio no garantiza el optimo global en Mochila 0-1.

Aclaraciones tecnicas del proyecto

El flujo principal del proyecto esta concentrado en `src/main.cpp`. Desde este archivo se llaman los modulos de lectura, ordenamiento, busqueda, Mochila 0-1 y Kruskal. Los componentes principales del programa estan distribuidos en los archivos `parser`, `mergesort`, `binary_search`, `graph`, `kruskal` y `knapsack`.

Otra aclaracion importante es el comando de compilacion. Para evitar incluir `main.cpp` dos veces y causar un error de definicion duplicada de `main`, se recomienda compilar todos los archivos fuente una sola vez con:

```
g++ -std=c++17 -O2 -o ada_pf src/*.cpp
```

En el caso del MST, el orden de algunas aristas con pesos empatados puede variar dependiendo del ordenamiento, pero el peso total y la cantidad de aristas seleccionadas se mantienen consistentes. Lo importante para validar el resultado es que el arbol tenga `n - 1` aristas, no forme ciclos y alcance el peso total reportado.

Consideraciones de IA

Para el desarrollo de este informe se utilizaron herramientas de inteligencia artificial como apoyo en la organizacion de la informacion, revision de redaccion y estructuracion del documento. Sin embargo, el contenido fue revisado, ajustado y

validado por los integrantes del equipo con base en las implementaciones y los archivos de salida del trabajo.

La IA no reemplazo la implementacion ni la validacion tecnica del proyecto. Su uso se limito a apoyar la redaccion, mejorar la claridad de las explicaciones y ordenar los resultados en tablas y secciones coherentes con un informe academico.

Conclusiones

El trabajo permitio integrar varios temas del curso en un mismo flujo de ejecucion. Primero se construyo una base de datos en memoria a partir de un CSV real, luego se aplico ordenamiento y busqueda sobre esos datos, y finalmente se usaron los registros para formular problemas de grafos y programacion dinamica.

El Modulo A mostro la importancia de preparar correctamente los datos antes de aplicar algoritmos. El manejo de columnas por nombre, la validacion de campos vacios y la escritura de resultados permitieron que los demas modulos trabajaran sobre una estructura consistente.

El Modulo B permitio aplicar un enfoque codicioso con Kruskal y comprobar su funcionamiento mediante una verificacion manual en un subgrafo pequeno. La estructura Union-Find fue clave para detectar ciclos de forma eficiente y construir el MST sin revisar caminos completos en el grafo.

El Modulo C evidencio que no todos los problemas de optimizacion pueden resolverse con una regla codiciosa simple. La Mochila 0-1 requiere considerar combinaciones de items, y la programacion dinamica permite encontrar el optimo para la capacidad definida. Tambien se observo una limitacion practica: con `W = 500` y pesos basados en `TotalCharges`, ningun item seleccionado cabe en la mochila, por lo que el resultado optimo es vacio.

Como mejora futura, se podria revisar la escala usada para los pesos del Modulo C, ya que `TotalCharges` produce valores muy superiores a la capacidad. Otra posible mejora seria agregar pruebas automatizadas para validar el parser, el ordenamiento, la busqueda binaria, Kruskal y Mochila 0-1 con casos pequenos donde el resultado esperado sea facil de comprobar manualmente.

Bibliografia

IBM Sample Data Sets. WA_Fn-UseC_-Telco-Customer-Churn.csv. Dataset incluido en el repositorio del proyecto.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. Introduction to Algorithms. [completar referencia]

Material del curso Analisis y Diseno de Algoritmos, Universidad EAFIT. [completar referencia]

Todo este documento fue escrito en Notion